

Phase 1 — Supervised Learning — Regression

A complete study guide with concepts and fully solved mathematical examples

Start here. Regression predicts a number (a price, a temperature). Linear regression is the simplest possible model, so it's the perfect first algorithm to truly understand. This guide walks through five core methods — Linear, Polynomial, Ridge, Lasso, and Elastic Net — deriving each from first principles and solving a small numerical example by hand for every one.

Contents

1	Foundations: what regression actually is	1
2	Linear Regression REGRESSION	1
3	Polynomial Regression REGRESSION	3
4	Ridge Regression (L2) REGRESSION	4
5	Lasso Regression (L1) REGRESSION	5
6	Elastic Net REGRESSION	6
7	Putting it all together	6

1. Foundations: what regression actually is

In **supervised learning** we are given a training set of m examples, each a pair of an input $\mathbf{x}$ (the *features*) and an output y (the *target* / *label*):

$$\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(m)}, y^{(m)})\}.$$

In **regression**, the target $y \in \mathbb{R}$ is a continuous number (a price, a temperature, a height). The goal is to learn a function $h_{\boldsymbol{\theta}}(\mathbf{x})$ — the *hypothesis* — parameterised by weights $\boldsymbol{\theta}$, so that $h_{\boldsymbol{\theta}}(\mathbf{x}) \approx y$ for unseen inputs.

Key idea

We measure “how wrong” the model is with a **cost function** (also called a loss). *Training* means choosing $\boldsymbol{\theta}$ to make that cost as small as possible. Almost every method below differs only in (a) the shape of $h_{\boldsymbol{\theta}}$ and (b) what we add to the cost to discourage over-complex solutions.

Notation used throughout.

- m = number of training examples, n = number of features.
- $x_j^{(i)}$ = the j -th feature of the i -th example; $x_0^{(i)} = 1$ is the bias term.
- $\hat{y}^{(i)} = h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$ = the model's prediction.
- $X \in \mathbb{R}^{m \times (n+1)}$ = the *design matrix* (one row per example), $\mathbf{y} \in \mathbb{R}^m$ = the vector of targets.

2. Linear Regression

REGRESSION

Concept

Linear regression fits a straight line (or a flat hyperplane in higher dimensions) to the data. With a single feature:

$$\hat{y} = h_{\theta}(x) = \theta_0 + \theta_1 x,$$

where θ_0 is the *intercept* (value when $x = 0$) and θ_1 is the *slope* (how much $\hat{y}$ changes per unit of x). With many features it generalises to $\hat{y} = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n = \theta^T \mathbf{x}$.

The cost function (Mean Squared Error)

We penalise the squared gap between prediction and truth, averaged over the data:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x^{(i)} - y^{(i)})^2.$$

Squaring makes all errors positive and punishes large mistakes heavily; the $\frac{1}{2}$ is a convenience that cancels when we differentiate.

Solution 1 — the Normal Equation (closed form)

Because J is a convex (bowl-shaped) quadratic, its minimum is where the gradient is zero. Solving $\nabla_{\theta} J = 0$ gives a direct formula:

$$\theta = (X^T X)^{-1} X^T \mathbf{y}$$

For a single feature this reduces to the familiar least-squares lines:

$$\theta_1 = \frac{\sum_i (x^{(i)} - \bar{x})(y^{(i)} - \bar{y})}{\sum_i (x^{(i)} - \bar{x})^2} = \frac{S_{xy}}{S_{xx}}, \quad \theta_0 = \bar{y} - \theta_1 \bar{x}.$$

Solution 2 — Gradient Descent (iterative)

When $X^T X$ is too big to invert, we descend the cost surface step by step. The partial derivatives are

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_i (\hat{y}^{(i)} - y^{(i)}), \quad \frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_i (\hat{y}^{(i)} - y^{(i)}) x^{(i)},$$

and we repeatedly update, with *learning rate* α :

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j}.$$

Worked Example: fitting a line by hand

Data: $x = (1, 2, 3, 4, 5)$, $y = (2, 4, 5, 4, 5)$, so $m = 5$.

Step 1 — means. $\bar{x} = \frac{1+2+3+4+5}{5} = 3$, $\bar{y} = \frac{2+4+5+4+5}{5} = 4$.

Step 2 — the sums of squares.

$x^{(i)}$	$y^{(i)}$	$x - \bar{x}$	$y - \bar{y}$	$(x - \bar{x})(y - \bar{y})$
1	2	-2	-2	4
2	4	-1	0	0
3	5	0	1	0
4	4	1	0	0
5	5	2	1	2

$$S_{xx} = \sum (x - \bar{x})^2 = 4 + 1 + 0 + 1 + 4 = 10, \quad S_{xy} = 4 + 0 + 0 + 0 + 2 = 6.$$

Step 3 — coefficients.

$$\theta_1 = \frac{S_{xy}}{S_{xx}} = \frac{6}{10} = 0.6, \quad \theta_0 = \bar{y} - \theta_1 \bar{x} = 4 - 0.6 \times 3 = 2.2.$$

$$\hat{y} = 2.2 + 0.6x$$

Step 4 — predictions and error.

x	y	$\hat{y} = 2.2 + 0.6x$	$(\hat{y} - y)^2$
1	2	2.8	0.64
2	4	3.4	0.36
3	5	4.0	1.00
4	4	4.6	0.36
5	5	5.2	0.04

$$\text{SSE} = 2.4, \quad \text{MSE} = \frac{2.4}{5} = 0.48.$$

Step 5 — one gradient-descent step (starting at $\theta_0 = \theta_1 = 0$, $\alpha = 0.01$). All predictions are 0, so errors $\hat{y} - y = (-2, -4, -5, -4, -5)$.

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{5}(-2 - 4 - 5 - 4 - 5) = -4, \quad \frac{\partial J}{\partial \theta_1} = \frac{1}{5}[(-2)(1) + \dots + (-5)(5)] = \frac{-66}{5} = -13.2.$$

$$\theta_0 \leftarrow 0 - 0.01(-4) = 0.04, \quad \theta_1 \leftarrow 0 - 0.01(-13.2) = 0.132.$$

Both weights moved toward the exact answer (2.2, 0.6). Repeating this update thousands of times converges to the same line found in Step 3.

3. Polynomial Regression

REGRESSION**Concept**

A straight line cannot bend. Polynomial regression lets the model follow curves by adding *powers* of the original feature as new features:

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_d x^d.$$

The crucial insight: this is **still linear regression**, just on transformed features $x_1 = x$, $x_2 = x^2$, $\dots$, $x_d = x^d$. The model is linear in the *parameters* θ_j , so the exact same normal equation $\boldsymbol{\theta} = (X^\top X)^{-1} X^\top \mathbf{y}$ applies — we only change what goes into X .

Worked Example: exact quadratic fit through three points

Data: $(0, 1)$, $(1, 0)$, $(2, 3)$. Fit $\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2$.

With 3 points and 3 unknowns, the curve passes exactly through every point. Plugging each point in:

$$x = 0 : \quad \theta_0 = 1$$

$$x = 1 : \quad \theta_0 + \theta_1 + \theta_2 = 0$$

$$x = 2 : \quad \theta_0 + 2\theta_1 + 4\theta_2 = 3$$

From the first line $\theta_0 = 1$. Substitute into the others:

$$\theta_1 + \theta_2 = -1, \quad 2\theta_1 + 4\theta_2 = 2 \Rightarrow \theta_1 + 2\theta_2 = 1.$$

Subtracting the equations: $(\theta_1 + 2\theta_2) - (\theta_1 + \theta_2) = 1 - (-1) \Rightarrow \theta_2 = 2$, then $\theta_1 = -1 - \theta_2 = -3$.

$$\hat{y} = 2x^2 - 3x + 1$$

Check $x = 2$: $2(4) - 3(2) + 1 = 8 - 6 + 1 = 3 \checkmark$. A line could never hit all three of these points; the quadratic does so perfectly.

The dark side: overfitting and the bias–variance trade-off

Raising the degree d always reduces *training* error — a degree- $(m-1)$ polynomial can pass through every one of m points exactly. But such a curve wiggles wildly between points and predicts terribly on new data. This is **overfitting**.

Key idea

Bias = error from a model too simple to capture the pattern (e.g. a line for curved data) — *underfitting*. **Variance** = sensitivity to the particular training sample; a high-degree polynomial has high variance — *overfitting*. Good generalisation balances the two. The regularised methods in the next sections are the standard cure for high variance.

4. Ridge Regression (L2)**REGRESSION****Concept**

Ridge keeps the linear model but adds an **L2 penalty** — the sum of squared weights — to the cost. Large coefficients are now expensive, so the optimiser is pushed toward smaller, smoother weights. This is **regularization**: trading a little training accuracy for much better generalisation.

$$J_{\text{ridge}}(\boldsymbol{\theta}) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2}_{\text{fit the data}} + \underbrace{\lambda \sum_{j=1}^n \theta_j^2}_{\text{shrink the weights}}.$$

The hyperparameter $\lambda \geq 0$ controls the strength: $\lambda = 0$ gives ordinary least squares; $\lambda \rightarrow \infty$ drives all weights toward 0. (The intercept θ_0 is conventionally left unpenalised.)

Closed-form solution

Setting the gradient to zero gives a beautifully small change to the normal equation — just add λ down the diagonal:

$$\boldsymbol{\theta} = (X^T X + \lambda I)^{-1} X^T \mathbf{y}$$

Adding λI also guarantees the matrix is invertible even when features are correlated, which is a second practical benefit of Ridge.

Worked Example: coefficient shrinkage

Data (single feature, no intercept): $x = (1, 2, 3)$, $y = (1, 2, 2)$.

The two quantities we need:

$$z = X^T X = \sum x_i^2 = 1 + 4 + 9 = 14, \quad \rho = X^T \mathbf{y} = \sum x_i y_i = 1 + 4 + 6 = 11.$$

Here the closed form is the scalar $\theta = \frac{\rho}{z + \lambda} = \frac{11}{14 + \lambda}$. Sweeping λ :

λ	$\theta = \frac{11}{14+\lambda}$	comment
0	0.786	ordinary least squares
1	0.733	slightly shrunk
10	0.458	noticeably shrunk
14	0.393	halved
100	0.096	nearly (but never) zero

Key observation: the coefficient *shrinks smoothly toward zero* as λ grows but *never reaches exactly zero*. Ridge keeps every feature, only making each one quieter.

5. Lasso Regression (L1)

REGRESSION

Concept

Lasso (*Least Absolute Shrinkage and Selection Operator*) replaces the squared penalty with the **L1 penalty** — the sum of *absolute* weights:

$$J_{\text{lasso}}(\boldsymbol{\theta}) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \sum_{j=1}^n |\theta_j|.$$

This tiny change has a dramatic effect: Lasso can push coefficients to *exactly* zero, performing **automatic feature selection**. The surviving non-zero weights tell you which features the model actually relies on.

Soft-thresholding

$|\theta|$ is not differentiable at 0, so there is no simple matrix inverse. For one standardised feature the coordinate-descent update is the **soft-threshold** operator

$$\theta = \frac{S(\rho, \lambda)}{z}, \quad S(\rho, \lambda) = \text{sign}(\rho) \max(|\rho| - \lambda, 0).$$

The $\max(\cdot, 0)$ is what clips small coefficients all the way to zero.

Worked Example: a coefficient driven to exactly zero

Use the *same* data as the Ridge example: $z = 14$, $\rho = 11$. Now $\theta = \frac{\max(11 - \lambda, 0)}{14}$:

λ	$\theta = \frac{\max(11-\lambda, 0)}{14}$	comment
0	0.786	ordinary least squares
5	0.429	shrunk
10	0.071	almost gone
11	0.000	exactly zero — feature dropped
14	0.000	stays at zero

Contrast with Ridge: on identical data Ridge gave 0.393 at $\lambda = 14$ and never hit zero, whereas Lasso eliminates the feature completely at $\lambda = 11$. That is the difference between *shrinking* and *selecting*.

Why does L1 create sparsity? (the geometry)

Minimising the error subject to a penalty budget is like inflating the error ellipses until they touch the constraint region. The L2 region is a circle (smooth — first contact is almost never on an axis), while the L1 region is a diamond with sharp corners *on the axes*. The ellipse usually first touches a corner, where one coordinate is zero — hence sparse solutions.

6. Elastic Net**REGRESSION****Concept**

Elastic Net simply **adds both penalties** at once, inheriting the best of each:

$$J_{\text{enet}}(\boldsymbol{\theta}) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 + \lambda_1 \sum_j |\theta_j| + \lambda_2 \sum_j \theta_j^2.$$

The L1 part (λ_1) gives sparsity / feature selection; the L2 part (λ_2) gives stable shrinkage and a “grouping” effect that shares weight fairly among correlated features (which Lasso handles poorly — it tends to pick one of a correlated group arbitrarily).

Single-feature update

Combining the two derivations, the soft-threshold numerator (from L1) sits over a ridge-style enlarged denominator (from L2):

$$\theta = \frac{S(\rho, \lambda_1)}{z + \lambda_2}.$$

Worked Example: combining shrinkage and selection

Same data again: $z = 14$, $\rho = 11$. Take $\lambda_1 = 5$ and $\lambda_2 = 14$.

$$\theta = \frac{S(11, 5)}{14 + 14} = \frac{\max(11 - 5, 0)}{28} = \frac{6}{28} = 0.214.$$

Compare all three methods on the identical problem:

Method	θ	behaviour
Lasso ($\lambda_1 = 5$)	$6/14 = 0.429$	L1 shrink only
Ridge ($\lambda_2 = 14$)	$11/28 = 0.393$	L2 shrink only
Elastic Net (5, 14)	$6/28 = 0.214$	both — smaller, can still zero out

Elastic Net applies the L1 threshold (which would zero the weight once $\lambda_1 \geq \rho = 11$) *and* the L2 denominator shrinkage, giving the most conservative estimate while retaining the ability to perform selection.

7. Putting it all together

Side-by-side comparison

Method	Penalty added to MSE	What it does & when to use
Linear	none	Baseline straight-line fit. Use first; understand cost & gradient descent here.
Polynomial	none (adds $x^2, x^3, \dots$ features)	Captures curves. Watch for overfitting at high degree.
Ridge (L2)	$\lambda \sum \theta_j^2$	Shrinks all weights smoothly; never exactly zero. Great when many features are useful and correlated.
Lasso (L1)	$\lambda \sum \theta_j $	Drives some weights to exactly zero $\Rightarrow$ automatic feature selection. Great for sparse, interpretable models.
Elastic Net	$\lambda_1 \sum \theta_j + \lambda_2 \sum \theta_j^2$	Best of both: selection <i>and</i> stable shrinkage. Default choice with many correlated features.

Practical notes you should not skip

- **Standardise your features** (zero mean, unit variance) before Ridge/Lasso/Elastic Net. The penalties compare coefficient *sizes*, which is only fair when features share a scale.
- **Choose λ by cross-validation**, not by hand. Try a grid of values, measure validation error, keep the best. There is no single “correct” λ — it depends on data.
- **Don’t penalise the intercept.** It only sets the overall level of y and shouldn’t be shrunk toward zero.
- **More data fights overfitting** as effectively as regularization — often more so.

The one-sentence summary. Everything here is least-squares with the same cost $\frac{1}{2m} \sum (\hat{y} - y)^2$; Polynomial changes the *features*, while Ridge, Lasso, and Elastic Net change the *penalty* you add on top — squared weights (smooth shrinkage), absolute weights (sparse selection), or

both.

Companion Colab notebook implements every method above on real data with scikit-learn and from-scratch gradient descent, so you can reproduce these numbers and plot the regularization paths yourself.